Accurate but Inconsistent: When Neural Option Pricers Break No-Arbitrage Rules

DSCI-515 Statistical Learning | Course project proposal

Abstract

Neural networks can now price options and calibrate volatility models in milliseconds. The base paper for this project (Horvath, Muguruza and Tomas, 2021) shows that a small network can map five Heston model parameters to a full implied volatility surface. Such surrogates are judged by average pricing error. We ask a different question: when a surrogate is accurate on average, does it still break the rules that every valid option price must obey (no static arbitrage), and why? We reproduce the base paper's Heston surrogate, then train surrogates on Heston prices, whose ground truth is known and arbitrage-free. We hypothesize that (H1) violations concentrate where the learned surface has the wrong curvature in strike, even when pointwise error is tiny; (H2) violations also occur inside the training range and grow as training data shrink or label noise rises, so they reflect estimation variance and not only extrapolation; and (H3) supervising derivatives restores consistency at a smaller accuracy cost than penalising violations. We vary training size, label noise, architecture and loss, compare against ridge, gradient boosting and Gaussian process baselines, and report bias-variance decompositions with bootstrap confidence intervals. The expected insight is that a loss rewarding only price levels leaves the shape of the surface under-determined. If the networks prove robust, we will report that result and the limits of the evidence.

Introduction

The task. A European call option pays off if the asset ends above the strike. Its price depends on the strike, the maturity and the parameters of a market model. Under the Heston (1993) stochastic volatility model the price needs numerical integration, and calibrating the model to market quotes means repricing thousands of times. Horvath et al. (2021) move that cost offline: a network learns the map from the five Heston parameters to implied volatilities on a fixed strike and maturity grid, and calibration then runs against the fast network. Their code is public under an MIT license.

Why failure is plausible. Option prices are not arbitrary numbers. A valid call price surface must fall and be convex in strike, and rise with maturity, in forward terms (Carr and Madan, 2005). Convexity in strike is the implied probability density, so a violation means negative probability and a portfolio of options that earns money for free. The Heston prices used for training obey these rules, but a squared-error loss only scores price levels at sampled points. It says nothing about slopes or curvature, and a second derivative amplifies small errors in levels. A network can therefore be accurate in price yet wrong in shape, much as the retrieval example in the course guideline shows strong scores hiding a lexical shortcut.

Research question. When and why does a neural option pricer that is accurate on average violate no-arbitrage conditions? We test three hypotheses.

- **H1 (accuracy hides shape).** Violations concentrate where the curvature error of the learned surface is large, and curvature error predicts violations better than point wise error does.
- **H2 (variance, not only extrapolation).** Violations occur inside the training parameter range and grow as training size falls or label noise rises.
- **H3 (the loss decides).** Training on derivatives restores consistency at a smaller accuracy cost than adding a direct arbitrage penalty.

Scope and contributions. This is Option 1. Our manipulations sit mostly inside the training distribution (noise, data size, architecture, loss), because severe out-of-distribution tests only confirm what is already expected. We contribute a reproduction of the base paper, a controlled study that separates accuracy, shape and consistency, a comparison of two repairs, and an interactive demo for the class.

Motivation

Why it matters in practice. Surrogate pricers are attractive because they remove the calibration bottleneck for slow models. A surface with negative implied density can, however, destabilise calibration and distort hedge ratios. Ackerer et al. (2020) stress that implied volatility models must generate arbitrage-free prices and that a low training loss matters as much as generalisation. Itkin (2019) addressed no-arbitrage pricing with a trained network, which he noted the literature had so far ignored.

Why it matters for statistical learning. The setting is a clean test of a core idea in the course: a model optimises what the loss rewards, not what we care about. The ground truth is known and the property is checkable by formula, so we can measure failure without human labels, which is rare. We can also split error into bias and variance and see which one drives inconsistency.

Related work and gap. Three strands exist. (1) Neural pricing and calibration: Horvath et al. (2021) and the review by Ruf and Wang (2020), which covers more than a hundred papers. (2) Enforcing no-arbitrage: soft constraints on surface fits (Ackerer et al., 2020) and the caution raised by Itkin (2019). (3) Learning derivatives: differential machine learning trains on labels together with their derivatives (Huge and Savine, 2020). In the papers reviewed so far, arbitrage control appears as part of a fitting method or as a caveat. We did not find a controlled study of when an otherwise accurate surrogate, trained on model prices, becomes inconsistent, which factors drive it, and which repair works best. That is the gap, and our search so far has not been exhaustive.

Why the result is interesting either way. Frequent violations despite tiny average error would be a surprising and practical warning that accuracy benchmarks are not enough. If violations vanish inside the training range and appear only near the parameter bounds, we document where the method is safe. Either outcome gives guidance on how much data and which loss a consistent surrogate needs.

Experimental Setting

1. Reproduction. The authors' Heston notebook (NN-StochVol-Calibrations, MIT licence) was written for 2019-era Keras (it uses an old Keras import and a Windows-style file path), and we did not run it as published, so we ported it to PyTorch with the same data, split, scaling, architecture (five inputs, three ELU layers of width 30, 88 outputs), loss and optimiser. In a preliminary run on 1,800 held-out surfaces it reaches a mean relative error of 0.18 percent and an RMSE of 0.0009 in implied volatility (about 0.09 volatility points), with a worst case of 18 percent. The final validation error is consistent with the authors' own training log (0.0279 against 0.0298 in standardised units, one seed; validation error moves by about 0.002 between epochs), but we have not yet compared these figures with those reported in the paper. We will also study the worst cases, since they may show where violations cluster.

2. Data. Labels are Heston call prices from the characteristic-function formula with numerical integration, so ground truth is known to the integration tolerance. The authors' released training set holds only 12,000 parameter sets, so we write our own pricer to reach larger sizes and validate it against their file. We draw parameter sets with a Sobol sequence over the ranges in the reference code and compute implied volatilities on the base paper's 88-point grid (11 strikes from 0.5 to 1.5 by 8 maturities from 0.1 to 2.0), plus call prices on a denser grid, because the coarse grid is too sparse to check convexity reliably. Training sizes are 2k, 10k, 50k and 250k parameter sets. The test set has 20k fresh sets, including a boundary subset within 10 percent of the parameter bounds.

3. Models and baselines. (a) The base paper network and its variants. (b) Ridge regression on polynomial features of the parameters, with the degree chosen by cross-validation. (c) Gradient boosting and (d) a Gaussian process, both fitted to the leading principal components of the surface. The baselines show whether inconsistency is specific to neural networks or shared by any flexible regressor.

4. Manipulations. We change one factor at a time, plus a small crossed design for training size and label noise.

Factor	Levels	Purpose
Training size	2k, 10k, 50k, 250k	Isolate estimation variance (H2)
Label noise	0, 0.1, 0.5, 2 percent, multiplicative	Mimic Monte Carlo or market noise (H2)

Factor	Levels	Purpose
Representation	Grid output (base paper) or pointwise input (parameters, strike, maturity)	Pointwise nets expose strike curvature through autodiff (H1)
Capacity, activation	Depth 2 to 6, width 30 to 256; ELU, softplus, ReLU	ReLU pointwise nets have zero curvature almost everywhere (H1)
Loss	Squared error; plus derivative loss; plus arbitrage penalty	Compare the two repairs (H3)
Input region	Interior or near the bounds	Separate extrapolation from variance (H2)
Seeds	10 per setting	Variance and confidence intervals

5. Metrics.

- **Accuracy:** RMSE and 99th percentile error of implied volatility, in volatility points.
- **Shape:** relative error of strike curvature (the implied density), of maturity slope, and of the Jacobian with respect to the parameters, which drives calibration.
- **Consistency:** share of surfaces and of grid cells violating each Carr and Madan condition (decreasing and convex in strike, increasing in maturity), and the negative probability mass.
- **Statistics:** 95 percent bootstrap confidence intervals over test surfaces and seeds, a bias-variance decomposition from bootstrap-trained models, and the AUC of local error as a predictor of violations.

The checks are applied first to exact Heston surfaces, to set a tolerance with a near-zero false-positive rate.

6. Judging the hypotheses. H1 is supported if curvature error predicts violating cells with a higher AUC than pointwise error, with the bootstrap interval for the difference above zero; otherwise it is refuted. H2 is supported if the interior violation rate is clearly above the detector's false-positive rate and rises as data shrink and noise grows, with the variance term tracking it; it is refuted if violations appear only near the bounds. H3 is supported if derivative supervision reaches a target violation rate (for example under 1 percent of cells) with a smaller accuracy loss than the penalty, shown as an accuracy against violation frontier.

7. Class demo. A small web app serves the trained models. People move parameter sliders on their phones, compare exact, network and baseline surfaces, see violating regions highlighted, and try to find the setting that breaks the network. The demo illustrates the findings; the evidence comes from the experiments above.

8. Plan and risks. Weeks 1 to 2: data pipeline and pricer. Weeks 3 to 5: baselines and manipulations. Week 6: repairs and analysis. Week 7: demo and write-up. Risks: violations may be rare, in which case we report the attempts made and explain the robustness with the references; several hundred training runs are needed, so models stay small and only the largest runs need a GPU; and the base paper figures may differ from ours, which we will document rather than tune away.

References

Base paper: Horvath et al. (2021). Peer-reviewed: Ackerer et al., Carr and Madan, Heston, Ruf and Wang. Preprints: Huge and Savine, Itkin.

1. Ackerer, D., Tagasovska, N. and Vatter, T. (2020). Deep smoothing of the implied volatility surface. NeurIPS 2020. arXiv:1906.05065.
2. Carr, P. and Madan, D. (2005). A note on sufficient conditions for no arbitrage. Finance Research Letters, 2(3), 125-130.
3. Heston, S. L. (1993). A closed-form solution for options with stochastic volatility with applications to bond and currency options. Review of Financial Studies, 6(2), 327-343.

4. Horvath, B., Muguruza, A. and Tomas, M. (2021). Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. Quantitative Finance, 21(1), 11-27. arXiv:1901.09647. Code: github.com/amuguruza/NN-StochVol-Calibrations.
5. Huge, B. and Savine, A. (2020). Differential machine learning. arXiv:2005.02347 (preprint).
6. Itkin, A. (2019). Deep learning calibration of option pricing models: some pitfalls and solutions. arXiv:1906.03507 (preprint).
7. Ruf, J. and Wang, W. (2020). Neural networks for option pricing and hedging: a literature review. Journal of Computational Finance. DOI 10.21314/JCF.2020.390.